

The Ball and Beam System

EE 222 Course Project Part 1

Group Members: Ian Chuang, Reid Dye, Nima Rahmanian, Ahmed Shakil
UC Berkeley

<https://github.com/ian-chuang/EE222-Nonlinear-Systems-Ball-and-Beam-Project>

April 4, 2025

1. Controller Designs

1.1. Controller 1: Feedback Linearization + LQR

We implemented Feedback Linearization + LQR because it was shown to be an effective controller for the ball and beam example in [1].

In feedback linearization, the idea is to create a linear system around an output you want to control by designing a control that cancels out the nonlinearity of the system.

We start with our output function:

$$y = h(x),$$

where in this case, $h(x) = x_1$, the position of the ball. We then differentiate the output until the input appears. The number of times we differentiate being the relative degree r :

$$\begin{aligned}\dot{y} &= L_f h(x), \\ \ddot{y} &= L_f^2 h(x), \\ &\vdots \\ y^{(n)} &= L_f^n h(x) + L_g L_f^{n-1} h(x) u.\end{aligned}$$

From this, we perform a change of variables transformation:

$$\begin{aligned}z_1 &= y = h(x), \\ z_2 &= \dot{y} = L_f h(x), \\ &\vdots\end{aligned}$$

where if we set

$$u = \frac{-L_f^n h(x) + v}{L_g L_f^{n-1} h(x)},$$

the system is transformed into a linear system:

$$\begin{aligned}\dot{z}_1 &= z_2, \\ \dot{z}_2 &= z_3, \\ &\vdots \\ \dot{z}_n &= v.\end{aligned}$$

Finally, LQR is used to optimally place the poles of the linear system and solve for K in:

$$v = -Kz.$$

1.2. Controller 2: Extended/Time-Varying LQR

The FBL approach is nice because it ensures the system controlled by the LQR is in fact linear, so all the mathematical guarantees that the LQR gives are accurate. However, the linearized dynamics are different from the true dynamics, so our notion of optimality is incorrect. To correct this, we implemented an LQR directly on the nonlinear dynamics using a linearization about the target state. As we are only given the desired output and two derivatives, not the full state, we had to solve for the linearization point.

$$\begin{bmatrix} \dot{x}_d \\ \ddot{x}_d \end{bmatrix} - \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} f(x_d, \dot{x}_d, \underbrace{\theta, \dot{\theta}, u}_{\text{unknown}}) = 0 \quad (1)$$

Once we had solved the above equation for u , $\dot{\theta}$, and $\ddot{\theta}$, we could simply linearize the system about the computed target state, then solve the Riccati equations to find a locally valid LQR controller.

2. Observer Designs

2.1. Extended Luenberger Observer Algorithm

We utilize the Extended Luenberger Observer, first proposed in [2]. Here, we start with a brief overview of the Luenberger Observer, and discuss its extension to non-linear systems.

2.1.1 Luenberger Observer

Given a linear system in state-space form with known parameters

$$\begin{aligned}\dot{x} &= Ax + Bu \\ y &= Cx,\end{aligned}$$

one may design an observer to estimate the unknown state as follows

$$\begin{aligned}\dot{\hat{x}} &= A\hat{x} + Bu + L(y - \hat{y}), \\ \hat{y} &= C\hat{x}.\end{aligned}$$

With this particular design, the state-estimation error dynamics $e = x - \hat{x}$ may be shown to obey

$$\dot{e} = (A - LC)e$$

If the pair $[A, C]$ is observable, then we may choose L such that $e \rightarrow 0$ exponentially for any initial conditions $x(0), \hat{x}(0)$.

2.1.2 Extended Luenberger Observer (ELO)

Consider the nonlinear system with known parameters

$$\begin{aligned}\dot{x} &= f(x, u), \\ y &= h(x).\end{aligned}$$

Define

1. $A(\hat{x}, \hat{u}) := \frac{\partial f}{\partial x}|_{(\hat{x}, \hat{u})}$
2. $C(\hat{x}) := \frac{\partial h}{\partial x}|_{\hat{x}}$

We linearize the true system dynamics about the estimated system state

1. $\dot{x} \approx f(\hat{x}, u) + A(\hat{x}, \hat{u})(x - \hat{x})$ (we only model deviations to the state variable)
2. $y \approx C(\hat{x})(x - \hat{x})$

If we design the observer as below

$$\begin{aligned}\dot{\hat{x}} &= f(\hat{x}, u) + L(\hat{x}, \hat{u})(y - \hat{y}), \\ \hat{y} &= h(\hat{x}),\end{aligned}$$

we may show that the state-estimation error dynamics $e = x - \hat{x}$ obey

$$\dot{e} \approx (A(\hat{x}, \hat{u}) - L(\hat{x}, \hat{u})C(\hat{x}))e$$

If the system is locally observable (i.e. the pair $[A(\hat{x}, \hat{u}), C(\hat{x})]$ is observable in our domain of interest), then we may choose $L(\hat{x}, \hat{u})$ such that $e \approx \rightarrow 0$ exponentially so long as our linearization approximation is mild.

2.2. Extended Kalman Filter (EKF)

As a backup, we also instantiated the MATLAB implementation of the Extended Kalman Filter to have a black box backup observer. The EKF model [3] is as follows

$$\begin{aligned}x_{k+1} &= f(x_k, u_k, t) + w_k, \\ z_{k+1} &= h(x_k, t) + v_k,\end{aligned}$$

where x_k is the state, f is a nonlinear function, u_k is the control input, w_k is a zero-mean multivariate gaussian with a covariance (Q_k) we select, z_k is the measurements we are provided, h is a nonlinear function of the states which in our case is simply results in the first and third variables of the state vector, and finally v_k is another zero-mean multivariate gaussian with a covariance (R_k) we select. A complication we run into here is that the EKF in this form is a discrete-time algorithm, while

our model for the system is a continuous-time model. To rectify this we use Euler's method to discretize f as follows (note that f_{old} refers to the f from the original model of the system):

$$f := x_{k+1} = x_k + f_{old}(x_k, u_k)\Delta t.$$

The EKF algorithm has two stages, the first being the predict stage and the second being the update stage. During the prediction stage, the state is estimated using the previous control input and the previous state estimate. Then during the update stage, we use a number of calculations to predict an updated state estimate using the current measurements that we receive from the system. Below, we provide the equations for each stage:

Note that $\hat{x}_{n|m}$ represents the estimate of x at time n given observations up to and including time $m \leq n$.

Predict

$$\text{Predicted state estimate } \hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1}, u_{k-1})$$

$$\text{Predicted covariance estimate } P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q_{k-1}$$

Update

$$\text{Innovation or measurement residual } \tilde{y}_k = z_k - h(\hat{x}_{k|k-1})$$

$$\text{Innovation (residual) covariance } S_k = H_k P_{k|k-1} H_k^T + R_k$$

$$\text{Near-optimal Kalman gain } K_k = P_{k|k-1} H_k^T S_k^{-1}$$

$$\text{Updated state estimate } \hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k \tilde{y}_k$$

$$\text{Updated covariance estimate } P_{k|k} = (I - K_k H_k) P_{k|k-1}$$

where the state transition and observation matrices are defined as the following Jacobians:

$$F_k = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}_{k-1|k-1}, u_k} \quad \text{and} \quad H_k = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}_{k|k-1}}.$$

The EKF does not really give any hard guarantees about the convergence of its estimates. We tested the EKF by using it as an observer while trying to control the system. In figure 1, we see that it does a good job in producing its estimates. Despite the fact that noise is introduced into our calculations, the estimates stay close to our reference.

2.3. Moving Horizon Estimation (MHE)

In an effort largely driven by curiosity, we implemented a moving horizon estimator based on [4], an optimization-based observer that is essentially the dual of receding horizon control for the state estimation problem. We model the system with noise similarly to the EKF, though we use w_k as an argument to f instead of assuming additivity. Then, we formulate the optimization problem:

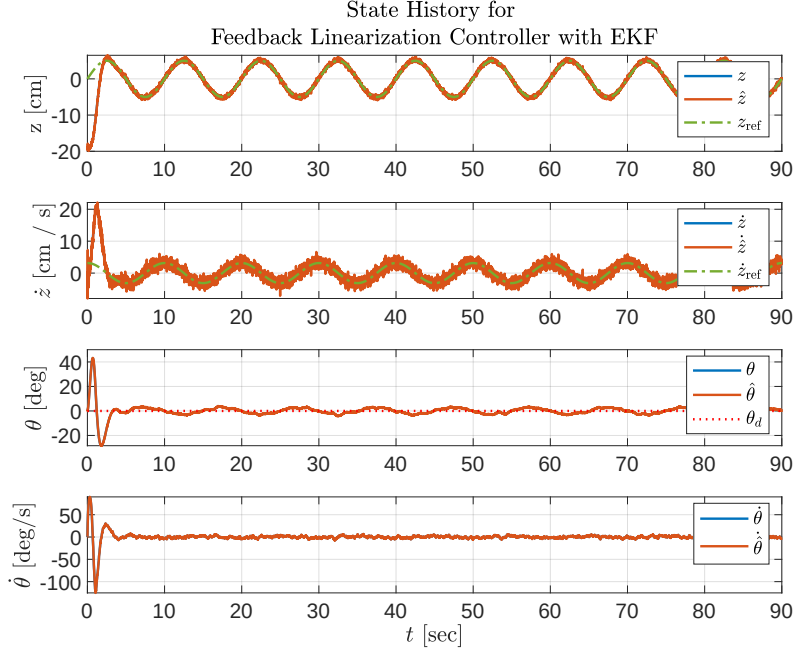


Figure 1: Comparison of Measured State and EKF Predicted State. The measured states are $z, \dot{z}, \theta, \dot{\theta}$, while the observer-predicted states are $\hat{z}, \hat{\dot{z}}, \hat{\theta}, \hat{\dot{\theta}}$.

$$\min_{x,w,v} (x_0 - \bar{x}_0)^\top P (x_0 - \bar{x}_0) + \sum_{k=0}^N w_k^\top Q w_k + v_k^\top R v_k \quad (2)$$

$$\text{s.t. } x_{k+1} = f(x_k, u_k, w_k) \quad \forall k \in \{0, \dots, N-1\} \quad (3)$$

$$x_k \in \mathcal{X} \quad " \quad (4)$$

where $\bar{x}_0$ is some a priori estimate for the system state at the start of the horizon and $\mathcal{X}$ is the physically reachable set of the state space. This moving horizon approximation to the full optimal estimation problem relies on the marginalization of past states to be computationally tractable - we approximate all of the history with the "cost-to-arrive" term involving the cost matrix P . At each timestep, we need to update this arrival cost by updating $\bar{x}_0$ and P . We do this using the discrete-time algebraic riccati equation (DARE):

$$\bar{x}_{0,k+1} = x_{1,k}^* \quad (5)$$

$$P_{k+1} = GQG^\top + AP_kA^\top + APC^\top(R + CPC^\top)^{-1}CPA^\top \quad (6)$$

where $A = \frac{\partial f_{cl}}{\partial x}$ is the linearized closed-loop dynamics, $C = \frac{\partial h}{\partial x}$ is the measurement model jacobian, and $G = \frac{\partial f}{\partial w}$ is the process noise jacobian. We need to wrap Q in G and $G^\top$ because Q is a weight on w_k , not x like in the LQR, so wrapping with G performs the change of variables - $GQG^\top$ is the equivalent state weight matrix (up to first order).

Table 1: Square Wave Overall Score

observer \ controller	<i>ELO</i>	<i>EKF</i>	<i>MHE</i>
<i>FBL</i>	3.37	3.43	3.47
<i>TV – LQR</i>	2.99	3.00	2.99

2.3.1 Numerical Implementation

We tested both a fourth- and second-order explicit Runge-Kutta method for formulating the discrete dynamics. We saw no significant difference between the two, and thus chose the second-order method for its reduced complexity. To help the optimization run faster, we ordered our variables and constraints to ensure a banded and nearly block diagonal structure. This allows sparse matrix solvers to efficiently solve the constraints, which significantly improved runtime.

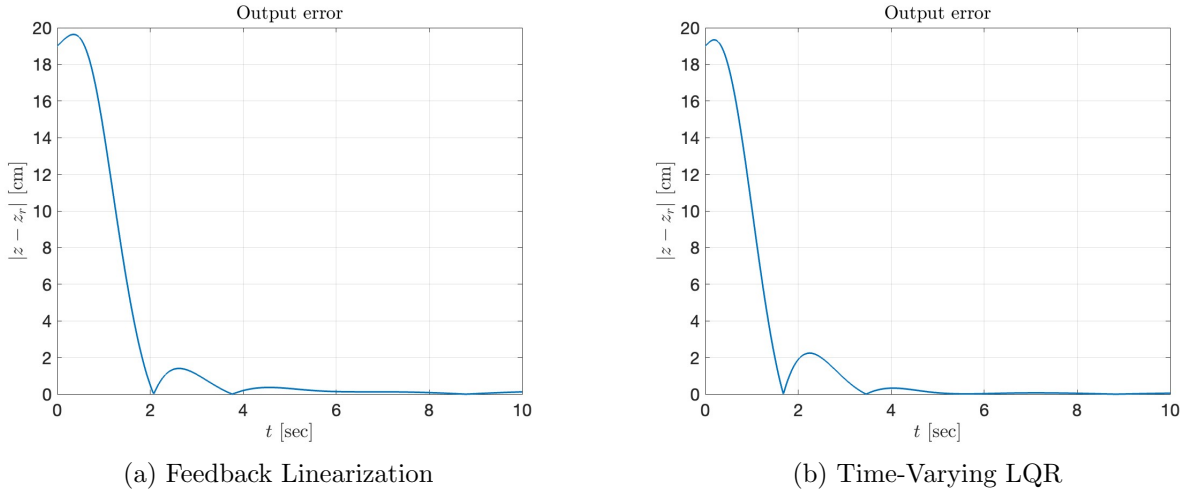


Figure 2: Ball position error for different controllers with Extended Luenberger Observer

3. Simulation Results

We tested both Feedback Linearization and Time-Varying LQR controllers on the ball-and-beam simulation with the Extended Luenberger Observer (ELO). The ball starts at the end of the beam and must track a sinusoidal reference trajectory centered at the middle of the beam. The tracking/output errors of both controllers are shown in 2.

Overall, both controllers perform well, with minimal overshoot occurring between $t = 2$ and $t = 4$ seconds. Both settle around $t = 4$ seconds with near-zero steady-state error. However, it is important to note that the simulation does not include noise.

Additionally, we compare and plot the ELO-predicted state against the actual state in 3. The ELO closely tracks the true state, though this is expected in a noise-free simulation.

Finally, we gathered the scores for all combinations of controllers and observers on the square wave task. The final scores are tabulated in table 1.

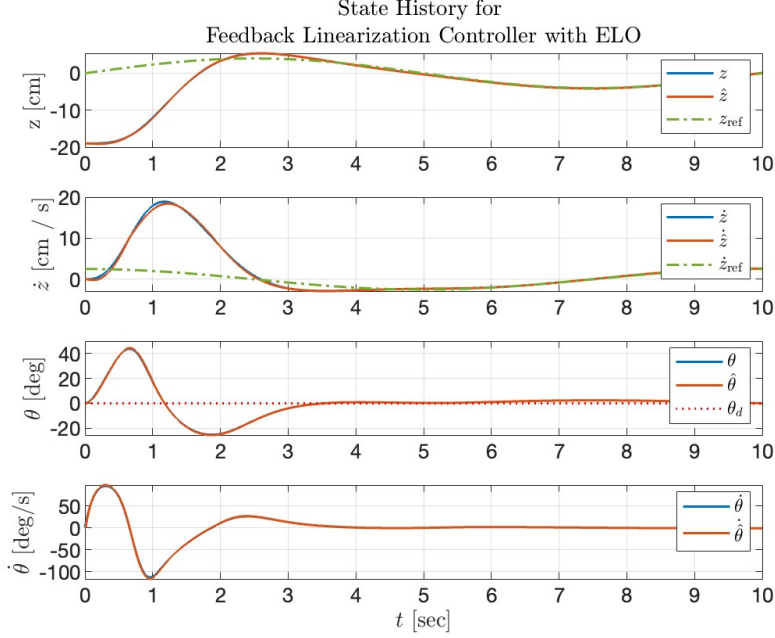


Figure 3: Comparison of Measured State and Extended Luenberger Observer Predicted State. The measured states are $z, \dot{z}, \theta, \dot{\theta}$, while the observer-predicted states are $\hat{z}, \hat{\dot{z}}, \hat{\theta}, \hat{\dot{\theta}}$.

4. Discussion

4.1. Feedback Linearization Considerations

In the case of the ball and beam dynamics, the relative degree is not well-defined. After following the procedure described in section 1.1 for the ball and beam dynamics, $L_g L_f^{n-1} h(x)$ is zero when the motor angular velocity or ball position are zero, making feedback linearization inapplicable. However, in the case of the ball and beam, the dynamics can be approximated by removing terms that include the input since these terms follow conditions explained in [1]. With the input terms removed the output can be further differentiated, allowing for a well-defined relative degree of 4.

4.2. Extended LQR Practical Considerations

The need for a numerical rootfinding solve followed by a Riccati solve at every timestep means that the extended LQR is noticeably slower than feedback linearization to run. When implemented on a real system, this means more unmodeled delay in the feedback loop, which could cause oscillations and even instability. We do not think this will be a significant issue, as computation time was less than 2ms, but it was at least four times slower than feedback linearization.

4.3. ELO Practical Considerations

In simulation, we found that observer poles of $[-4, -4.5, -5, -35.5]$ on the state $[y, \dot{y}, \theta, \dot{\theta}]$ works particularly well. When the poles are less extreme (i.e. less negative), our state estimate may not converge to the true state estimation quickly enough, which would cause our tracking controller to fail. We further noticed that our tracking results were quite sensitive to early errors in $\dot{\theta}$, which explains why the observer gain on $\dot{\theta}$ is notably higher than the gains on the other state.

4.4. Observer Results

We found that all observers performed equally well, but this was likely because the simulation was perfect, so observers did not have to do much work to produce a good estimate. We hypothesize that as soon as we test on the real system with sensor and process noise and potentially model mismatch, will have to both re-tune all of our observers, and will likely see much more pronounced differences in their performance.

4.5. MHE Results

We found that MHE consistently performed very similarly to the EKF, which is unsurprising considering that the special case where $N = 1$ gives rise to an EKF where the optimization problem is fully solved at each timestep (instead of just performing a single step of Newton’s method). The main issue with MHE is the computation time - performing the NLP solve took around 6 ms, which could be enough to cause significant issues when testing on hardware.

5. Conclusion

Overall, the design and testing process went smoothly. Once we correctly implemented the controller tuning, the task became straightforward. However, since we conducted the experiment in a noise-free simulation, we recognize that controller tuning in a real-world scenario will be more challenging. For the real-world experiment, we plan to investigate how noise affects controller performance and tuning, potentially modify our dynamics to account for disturbances such as friction, and evaluate which controller-observer combination performs best.

References

- [1] J. Hauser, S. Sastry, and P. Kokotovic, “Nonlinear control via approximate input-output linearization: The ball and beam example,” *IEEE transactions on automatic control*, vol. 37, no. 3, pp. 392–398, 1992.
- [2] M. Zeitz, “The extended luenberger observer for nonlinear systems,” *Systems Control Letters*, vol. 9, no. 2, pp. 149–156, 1987.
- [3] W. contributors, “Extended kalman filter — Wikipedia, the free encyclopedia,” 2024. [Online; accessed 3-April-2024].
- [4] C. Rao, J. Rawlings, and D. Mayne, “Constrained state estimation for nonlinear discrete-time systems: stability and moving horizon approximations,” *IEEE Transactions on Automatic Control*, vol. 48, no. 2, pp. 246–258, 2003.